

Aufgabenblatt 7

Aufgabe 7.1

i) TBB

```
Time scalar:      74.7418 ms
Time Vc:          21.934 ms, speedup factor: 3.40757
Time timerITBB:   8.07905 ms, speedup factor: 9.25131
Vc:
Parallel and scalar results are the same.
TBB:
Parallel and scalar results are the same.
```

matrix

Die reine Vektorisierung mit Vc bringt bereits einen Speedup von ca. 3.5 gegenüber der skalaren Variante. Dies liegt sehr nahe am theoretischen Maximum von 4x, was durch die Nutzung von SIMD-Instruktionen erklärbar ist. Hier zeigt sich, dass die SIMD-Optimierung sehr effizient arbeitet.

Durch die zusätzliche Parallelisierung mit TBB konnte ein Gesamtspeedup von ca. 9.3 erreicht werden. Theoretisch wäre bei perfekter Auslastung aller 16 logischen Kerne (bei 8 physischen Kernen mit Hyperthreading) ein Speedup von bis zu 16 möglich. Die gemessenen ca. 9.3 deuten darauf hin, dass die Last gut verteilt wird, aber einige Faktoren wie Overhead und Speicherbandbreite eine vollständige Ausnutzung verhindern.

ii) OpenMP

```
Time scalar:      75.5939 ms
Time OpenMP:       7.58505 ms, speedup factor: 9.96618
Time timerITBB:    7.2341 ms, speedup factor: 10.4497
OpenMP:
Parallel and scalar results are the same.
TBB:
Parallel and scalar results are the same.
```

matrix2

Auch mit OpenMP wird ein Speedup von nahezu 10x erreicht. Die Kombination aus Vektorisierung (SIMD) und Thread-basiertem Parallelismus zeigt, dass beide Technologien gut zusammenspielen. OpenMP bietet dabei eine einfache Möglichkeit, parallele Schleifen zu schreiben, während die Vc-Bibliothek effizient die SIMD-Instruktionen nutzt.

Im direkten Vergleich zeigt sich, dass sowohl TBB als auch OpenMP sehr ähnliche Performance liefern. Der geringe Unterschied kann durch Scheduling, Overhead oder unterschiedliche Lastverteilungen erklärt werden.

Aufgabe 7.3 Counting Zeros

```
Scalar counter:      489566      Time: 51.5089
TBB atomic counter:  489566      Time: 15.5921
TBB mutex counter:   489566      Time: 47.4601
```

i) Atomic Counter

Beim Einsatz mehrerer Threads besteht die Gefahr von race conditions, wenn mehrere Threads gleichzeitig auf dieselbe Variable zugreifen und sie verändern möchten. In unserem Fall wollen alle Threads den gemeinsamen Zähler inkrementieren, sobald ein Nullwert gefunden wird.

Durch den Einsatz von `std::atomic<int>` wird sichergestellt, dass jede Inkrement-Operation atomar, also unteilbar, durchgeführt wird. So wird garantiert, dass keine Updates verloren gehen. Hier wird die Methode `fetch_add(1, std::memory_order_relaxed)` verwendet.

Relaxed memory ordering genügt, da wir nur zählen und keine Abhängigkeiten zwischen den Threads entstehen.

Atomare Operationen sind sehr effizient, da sie hardwareunterstützt direkt von der CPU ausgeführt werden.

Im Vergleich zur seriellen Variante (51.5 ms) konnte durch die Parallelisierung mit dem atomaren Counter die Laufzeit auf 15.6 ms reduziert werden. Dies ergibt einen Speedup von etwa 3.3x.

Der atomare Zugriff funktioniert hier sehr gut, da der kritische Abschnitt (reines Inkrement) sehr klein ist und somit die Synchronisation kaum Zeit kostet.

ii) Mutex

Statt des atomaren Counters kann man auch einen `tbb::spin_mutex` verwenden, um den Zugriff auf den Zähler `counterParM` abzusichern.

Der `spin_mutex` blockiert andere Threads so lange, bis der aktuelle Thread den Zähler fertig erhöht hat. Das Warten passiert dabei aktiv (busy waiting), was bei sehr kurzen Abschnitten wie hier oft schneller ist als ein normaler mutex, bei dem der Thread schlafen gelegt und später neu gestartet werden müsste.

Damit man den `spin_mutex` nicht jedes Mal manuell wieder freigeben muss, verwendet man den `scoped_lock`. Dieser sperrt automatisch beim Betreten des Blocks und gibt den Lock wieder frei, wenn der Block verlassen wird. Man braucht also kein `unlock()`.

Der `spin_mutex` wird global deklariert, weil alle Threads denselben Lock benutzen müssen, damit sie sich beim Erhöhen des Zählers gegenseitig nicht stören.

Im Vergleich zur seriellen Variante brachte die Nutzung des mutex nur eine kleine Beschleunigung (etwa 1.08x), weil sich die Threads trotzdem oft gegenseitig beim Zugriff auf den Zähler blockieren.

Das liegt daran, dass beim Zugriff auf den Mutex immer noch Wartezeiten entstehen, wenn mehrere Threads gleichzeitig inkrementieren wollen. In diesem Beispiel ist die atomare Variante also deutlich überlegen.